

Python Data Structures — Syntax Cheat Sheet

Creation + core operations for every DSA-relevant structure, at a glance

Quick index: Array/List · Stack · Queue · Deque · Linked List · Doubly Linked List · Dict/Map · Set · Sorted Map/Set · Heap · Tuple · String · Counter · Binary Tree · Graph

Array / List

list — dynamic array, most common structure

```
# Create
a = []
a = [1, 2, 3]
a = [0] * 5           # [0,0,0,0,0]
a = list(range(5))    # [0,1,2,3,4]
matrix = [[0]*3 for _ in range(3)] # 2D grid

# Add
a.append(x)            # add to end
a.insert(i, x)         # insert at index
a.extend([1,2])        # add multiple

# Remove
a.pop()               # remove & return last
a.pop(i)              # remove & return at i
a.remove(x)           # remove first match by value
del a[i]              # delete at index
a.clear()

# Access / search
a[0]; a[-1]           # first, last
a[1:3]                # slice
x in a                 # O(n) membership
a.index(x)             # first index of x
len(a)

# Sort / reverse
a.sort()              # in place, ascending
a.sort(reverse=True)
a.sort(key=lambda x: x[1])
sorted(a)             # returns new list
a.reverse()
a[::-1]               # reversed copy
```

Stack (LIFO)

Use plain list — append/pop from the end

```
# Create
stack = []

# Push
stack.append(x)

# Pop
top = stack.pop()

# Peek
top = stack[-1]

# Is empty
if not stack: ...

# Example: valid parentheses
for ch in s:
    if ch == '(':
        stack.append(ch)
    elif ch == ')':
        if not stack: return False
        stack.pop()
```

Queue (FIFO)

collections.deque — never use list.pop(0), it's O(n)

```
from collections import deque

# Create
q = deque()
q = deque([1,2,3])

# Enqueue (add to back)
q.append(x)

# Dequeue (remove from front)
front = q.popleft()

# Peek front
front = q[0]

# Is empty
if not q: ...

# Example: BFS
q = deque([start])
while q:
    node = q.popleft()
    for nb in graph[node]:
        q.append(nb)
```

Double-Ended Queue (Deque)

collections.deque — O(1) at both ends

```
from collections import deque

# Create
dq = deque([1,2,3])
dq = deque(maxlen=5) # bounded deque (sliding window)

# Add
dq.append(x)          # add right
dq.appendleft(x)      # add left

# Remove
dq.pop()              # remove right
dq.popleft()          # remove left

# Access
dq[0]; dq[-1]         # front, back

# Rotate
dq.rotate(1)          # right rotate by 1
dq.rotate(-1)         # left rotate by 1
```

Singly Linked List

No built-in — define a Node class manually

```
class Node:
    def __init__(self, val, next=None):
        self.val = val
        self.next = next

# Build: 1 -> 2 -> 3
head = Node(1, Node(2, Node(3)))

# Traverse
cur = head
while cur:
    print(cur.val)
    cur = cur.next

# Insert at head
new_head = Node(x, head)

# Insert after a node
node.next = Node(x, node.next)

# Delete node.next
node.next = node.next.next

# Reverse a linked list (prev = new head)
prev = None
while head:
    nxt = head.next
    head.next = prev
    prev, head = head, nxt
```

Doubly Linked List

Manual class with prev + next pointers

```
class DNode:
    def __init__(self, val):
        self.val = val
        self.prev = None
        self.next = None

# Insert x after node
def insert_after(node, x):
    new = DNode(x)
    new.next = node.next
    new.prev = node
    if node.next:
        node.next.prev = new
    node.next = new

# Delete a node
def delete(node):
    if node.prev:
        node.prev.next = node.next
    if node.next:
        node.next.prev = node.prev

# Note: OrderedDict often simulates
# a DLL for O(1) LRU cache lookups
```

Hash-based structures

Dictionary / Map

dict — hash map, insertion-ordered since 3.7

```
# Create
d = {}
d = {'a': 1, 'b': 2}
d = dict(a=1, b=2)
```

Set

set — hash set, unordered, unique elements

```
# Create
s = set()
s = {1, 2, 3}
s = set([1, 2, 2, 3]) # dedupes -> {1,2,3}
```

```
# insert / update
d['key'] = value

# Access
d['key']          # KeyError if missing
d.get('key')      # None if missing
d.get('key', 0)   # default if missing

# Check / delete
'key' in d
del d['key']
d.pop('key')      # remove & return
d.pop('key', None) # safe remove

# Iterate
for k in d: ...
for k, v in d.items(): ...
for v in d.values(): ...

# Build frequency map
freq = {}
for x in arr:
    freq[x] = freq.get(x, 0) + 1

# Or with defaultdict
from collections import defaultdict
freq = defaultdict(int)
for x in arr: freq[x] += 1

graph = defaultdict(list) # adjacency list
```

```
# Add / remove
s.add(x)
s.remove(x)      # KeyError if missing
s.discard(x)     # safe, no error

# Check
x in s           # O(1)

# Set algebra
a | b           # union
a & b           # intersection
a - b           # difference
a ^ b           # symmetric difference

# Dedupe a list
unique = list(set(arr)) # order not preserved
unique = list(dict.fromkeys(arr)) # order preserved

# frozenset - immutable, hashable
fs = frozenset([1,2,3]) # usable as dict key
```

Sorted Map / Set

pip install sortedcontainers — closest to C++ std::map/set

```
from sortedcontainers import SortedList, SortedDict, SortedSet

# SortedList - most commonly used in interviews
sl = SortedList([3, 1, 2])
sl.add(x)          # O(log n) insert, stays sorted
sl.remove(x)       # O(log n)
sl[0]; sl[-1]      # min, max
sl.bisect_left(x)  # index where x would go

# SortedDict - sorted by key
sd = SortedDict({3:'c', 1:'a'})
sd.peekitem(0)     # smallest key-value pair
sd.peekitem(-1)    # largest

# No sortedcontainers? Use heapq + bisect
import bisect
a = [1, 3, 5]
bisect.insort(a, 4) # insert keeping sorted
i = bisect.bisect_left(a, 4) # binary search position
```

Priority & ordering structures

Heap / Priority Queue

heapq — min-heap only, built into standard library

```
import heapq

# Create
h = []
heapq.heapify(h) # convert list in-place, O(n)

# Push / pop (min-heap)
heapq.heappush(h, x)
smallest = heapq.heappop(h)
smallest = h[0] # peek without removing

# Max-heap trick: negate values
heapq.heappush(h, -x)
largest = -heapq.heappop(h)

# Push tuples for priority + data
heapq.heappush(h, (priority, x))

# N largest / smallest
heapq.nlargest(3, arr)
heapq.nsmallest(3, arr)

# Example: top-k frequent
heapq.nlargest(k, freq.items(), key=lambda p: p[1])
```

Tuple

tuple — immutable, hashable (usable as dict/set key)

```
# Create
t = (1, 2, 3)
t = 1, 2, 3 # parens optional
single = (1,) # comma needed for 1-elem!

# Unpack
a, b, c = t
a, *rest = t # a=1, rest=[2,3]

# Use as dict/set key (coords, pairs)
seen = set()
seen.add((r, c))
visited = {}
visited[(x, y)] = True

# Swap using tuples
a, b = b, a
```

String

str — immutable sequence of characters

```
# Create / build
s = "hello"
chars = list(s) # -> ['h','e','l','l','o']
s2 = ''.join(chars) # chars -> string

# Common methods
s.lower(); s.upper()
s.strip() # trim whitespace
s.split(',') # -> list
s.replace('a', 'b')
s.find(sub) # -1 if not found
s.startswith('he')
s.isalpha(); s.isdigit()

# Slicing
s[::-1] # reverse
s[1:4] # substring

# Char <-> ASCII
ord('a') # 97
chr(97) # 'a'
```

Counter

collections.Counter — dict subclass for frequency counting

```
from collections import Counter

# Create (auto-counts)
c = Counter([1,1,2,3,3,3]) # {3:3, 1:2, 2:1}
c = Counter("aabbcc")

# Access (0 if missing, no KeyError)
c['x'] # 0, doesn't crash

# Most common
c.most_common(2) # [(elem, count), ...]

# Counter arithmetic
c1 + c2 # add counts
c1 - c2 # subtract (keeps positive only)

# Anagram check
Counter(s1) == Counter(s2)
```

Tree & graph representations

Binary Tree

No built-in — define a `TreeNode` class manually

Graph

Adjacency list via dict or defaultdict(list)

```

class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# Build small tree
root = TreeNode(1, TreeNode(2), TreeNode(3))

# DFS - recursive inorder
def inorder(node):
    if not node: return []
    return inorder(node.left) + [node.val] + inorder(node.right)

# BFS - level order
from collections import deque
q = deque([root])
while q:
    node = q.popleft()
    if node.left: q.append(node.left)
    if node.right: q.append(node.right)

```

```

from collections import defaultdict

# Build adjacency list
graph = defaultdict(list)
graph[u].append(v)
graph[v].append(u) # if undirected

# DFS (recursive)
visited = set()
def dfs(node):
    if node in visited: return
    visited.add(node)
    for nb in graph[node]:
        dfs(nb)

# BFS
q = deque([start])
visited = {start}
while q:
    node = q.popleft()
    for nb in graph[node]:
        if nb not in visited:
            visited.add(nb)
            q.append(nb)

```